

Министерство науки и высшего образования РФ  
Федеральное государственное автономное  
образовательное учреждение высшего образования  
«**СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ**»  
Институт космических и информационных технологий  
Кафедра прикладной информатики

## ОТЧЕТ ОБ УЧЕБНОЙ ПРАКТИКЕ

Кафедра прикладной информатики ИКИТ СФУ  
место прохождения практики  
Разработка приложения "Управление библиотекой"  
тема

Руководитель от университета

\_\_\_\_\_  
подпись, дата

П. П. Дьячук  
инициалы, фамилия

Студент КИ25-21Б,032537942  
номер группы, зачетной книжки

\_\_\_\_\_  
подпись, дата

Д. А. Куренкова  
инициалы, фамилия

Красноярск 2026

## СОДЕРЖАНИЕ

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                                                     | 3  |
| ОСНОВНАЯ ЧАСТЬ.....                                                               | 4  |
| 1. Создать графический интерфейс пользователя с использованием Windows Forms..... | 4  |
| 2. Добавить необходимые классы .....                                              | 5  |
| 3. Создать второе окно для редактирования или добавления книги .....              | 9  |
| 4. Реализовать редактирование, удаление, добавление книг в обоих окнах .          | 11 |
| 5. Реализовать сохранение актуальной версии списка книг в файл.....               | 13 |
| 6. Проверить программу на наличие ошибок, корректную работу исключений.....       | 16 |
| ЗАКЛЮЧЕНИЕ .....                                                                  | 18 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....                                             | 19 |

## ВВЕДЕНИЕ

**Тема:** Разработка приложения "Управление библиотекой".

**Цель:** создать приложение для управления библиотекой, которое позволит пользователям добавлять, редактировать и удалять книги, а также отслеживать информацию о них. Приложение должно использовать события, LinQ и визуальное программирование.

**Задание:**

Основные требования:

### 1) Функциональные требования:

- a) Пользователь должен иметь возможность добавлять новую книгу с указанием названия, автора, жанра и статуса (доступна, занята).
- b) Пользователь должен иметь возможность редактировать информацию о существующих книгах.
- c) Пользователь должен иметь возможность удалять книги из библиотеки.
- d) Приложение должно отображать список всех книг с возможностью фильтрации по статусу и жанру.
- e) Реализовать возможность сортировки книг по названию, автору или жанру.

Использовать события для обновления интерфейса при изменении списка книг (например, при добавлении, редактировании или удалении книги).

Реализовать сохранения актуальной версии списка книг в файл.

### 2) Использование LinQ:

Использовать возможности LinQ для фильтрации и сортировки книг.

Например, реализовать функции для поиска книг по ключевым словам в названии или авторе.

### 3) Визуальное программирование:

Создать графический интерфейс пользователя (GUI) с использованием Windows Forms или WPF. Интерфейс должен быть интуитивно понятным и удобным для пользователя.

### 4) Документация:

Оформить документацию, в которой описать структуру приложения, используемые технологии и библиотеки, а также инструкции по запуску и использованию приложения.

### 5) Дополнительные требования:

Приложение должно быть написано с учетом принципов ООП (объектно-ориентированного программирования).

Приложение должно быть протестировано на наличие ошибок (можно в ручном режиме) и корректно обрабатывать исключения.

## ОСНОВНАЯ ЧАСТЬ

### 1. Создать графический интерфейс пользователя с использованием Windows Forms

Необходимо создать приложение для управления библиотекой с графическим интерфейсом Windows Forms или WPF, было выбрано Windows Forms.

Создан проект «Приложение Windows Forms (Майкрософт)». В нём был создан графический интерфейс главного окна «Управление библиотекой», он изображен на рисунке 1.

|   | Id | Title                    | Author            | Genre  | IsAvailable                         |
|---|----|--------------------------|-------------------|--------|-------------------------------------|
| ▶ | 2  | Преступление и наказание | Федор Достоевский | Роман  | <input type="checkbox"/>            |
|   | 3  | Мастер и Маргарита       | Михаил Булгаков   | Роман  | <input checked="" type="checkbox"/> |
|   | 4  | Евгений Онегин           | Александр Пушкин  | Поэзия | <input checked="" type="checkbox"/> |
|   | 5  | Мёртвые души             | Николай Гоголь    | Роман  | <input type="checkbox"/>            |
|   | 6  | Убить пересмешника       | Харпер Ли         | Роман  | <input checked="" type="checkbox"/> |

Рисунок 1 – Главное окно

Таблица книг реализована через элемент DataGridView. Она разделена на столбцы: ID, Название, Автор, Жанр, Статус. Данные загружаются из коллекции книг через привязку к источнику данных. Редактирование происходит через отдельное окно.

Фильтрация по жанру и статусу выполняется через выпадающие списки (ComboBox). При выборе значения из списка таблица автоматически обновляется благодаря обработчикам событий SelectedIndexChanged.

Кнопки «Добавить», «Редактировать» и «Удалить» расположены на верхней панели формы. Кнопки «Редактировать» и «Удалить» активируются только после выбора строки в таблице, что предотвращает случайные действия с невыбранной книгой.

## 2. Добавить необходимые классы

Был создан класс Book, каждая книга содержит в Id (уникальный идентификатор), Title (название), Author (автор), Genre (жанр), IsAvailable (статус: доступна или занята).

```
public class Book
{
    Ссылка: 11
    public int Id { get; set; }
    Ссылка: 16
    public string Title { get; set; }
    Ссылка: 12
    public string Author { get; set; }
    Ссылка: 14
    public string Genre { get; set; }
    Ссылка: 11
    public bool IsAvailable { get; set; }

    Ссылка: 1
    public Book()
    {
        Id = 0;
        Title = string.Empty;
        Author = string.Empty;
        Genre = string.Empty;
        IsAvailable = true;
    }

    Ссылка: 6
    public Book(int id, string title, string author, string genre, bool isAvailable)
    {
        Id = id;
        Title = title;
        Author = author;
        Genre = genre;
        IsAvailable = isAvailable;
    }

    Ссылка: 0
    public override string ToString()
    {
        return $"{Title} - {Author} ({Genre}) {(IsAvailable ? "Доступна" : "Занята")}";
    }
}
```

Рисунок 2 – Класс Book

Класс LibraryManager управляет коллекцией книг. Он предоставляет методы для добавления, редактирования и удаления книг (рисунок 3), выполняет поиск и фильтрацию с помощью LINQ (рисунок 4), а также сохраняет и загружает данные в формате JSON (рисунок 5).

```

Ссылка: 1
public void AddBook(Book book)
{
    if (book == null)
        throw new ArgumentNullException(nameof(book), "Книга не может быть null");

    book.Id = _books.Count > 0 ? _books.Max(b => b.Id) + 1 : 1;
    _books.Add(book);
    SaveToFile();
    OnBooksChanged();
}

Ссылка: 1
public void EditBook(int id, string title, string author, string genre, bool isAvailable)
{
    Book book = _books.FirstOrDefault(b => b.Id == id);
    if (book == null)
        throw new Exception($"Книга с ID {id} не найдена");

    book.Title = title;
    book.Author = author;
    book.Genre = genre;
    book.IsAvailable = isAvailable;

    SaveToFile();
    OnBooksChanged();
}

Ссылка: 1
public void DeleteBook(int id)
{
    Book book = _books.FirstOrDefault(b => b.Id == id);
    if (book == null)
        throw new Exception($"Книга с ID {id} не найдена");

    _books.Remove(book);
    SaveToFile();
    OnBooksChanged();
}

```

Рисунок 3 – Методы добавления, редактирования и удаления книг

Эти методы выполняют основные операции с книгами. После каждого изменения вызывается `SaveToFile()` для сохранения данных и `OnBooksChanged()` для уведомления интерфейса.

```

Ссылка: 0
public List<Book> SearchBooks(string keyword)
{
    if (string.IsNullOrEmpty(keyword))
        return _books.ToList();

    keyword = keyword.ToLower();
    return _books
        .Where(b => b.Title.ToLower().Contains(keyword) ||
                  b.Author.ToLower().Contains(keyword) ||
                  b.Genre.ToLower().Contains(keyword))
        .ToList();
}

Ссылка: 0
public List<Book> FilterByStatus(bool isAvailable)
{
    return _books.Where(b => b.IsAvailable == isAvailable).ToList();
}

Ссылка: 0
public List<Book> FilterByGenre(string genre)
{
    if (string.IsNullOrEmpty(genre))
        return _books.ToList();

    return _books.Where(b => b.Genre.Equals(genre, StringComparison.OrdinalIgnoreCase)).ToList();
}

Ссылка: 0
public List<Book> SortBooks(string sortBy)
{
    switch (sortBy)
    {
        case "Название":
            return _books.OrderBy(b => b.Title).ToList();
        case "Автор":
            return _books.OrderBy(b => b.Author).ToList();
        case "Жанр":
            return _books.OrderBy(b => b.Genre).ToList();
        default:
            return _books.ToList();
    }
}

```

Рисунок 4 – Примеры LINQ-запросов для поиска, фильтрации и сортировки книг

```

Ссылка: 4
public void SaveToFile()
{
    try
    {
        string json = JsonSerializer.Serialize(_books, new JsonSerializerOptions { WriteIndented = true });
        File.WriteAllText(_saveFilePath, json);
    }
    catch (Exception ex)
    {
        throw new Exception($"Ошибка сохранения файла: {ex.Message}");
    }
}

Ссылка: 1
public void LoadFromFile()
{
    try
    {
        if (File.Exists(_saveFilePath))
        {
            string json = File.ReadAllText(_saveFilePath);
            var loaded = JsonSerializer.Deserialize<List<Book>>(json);
            if (loaded != null)
            {
                _books.Clear();
                foreach (var book in loaded)
                {
                    _books.Add(book);
                }
            }
        }
        else
        {
            AddTestBooks();
        }
    }
    catch (Exception)
    {
        _books.Clear();
        AddTestBooks();
    }
}

```

Рисунок 5 – Сохранение и загрузка данных в формате JSON

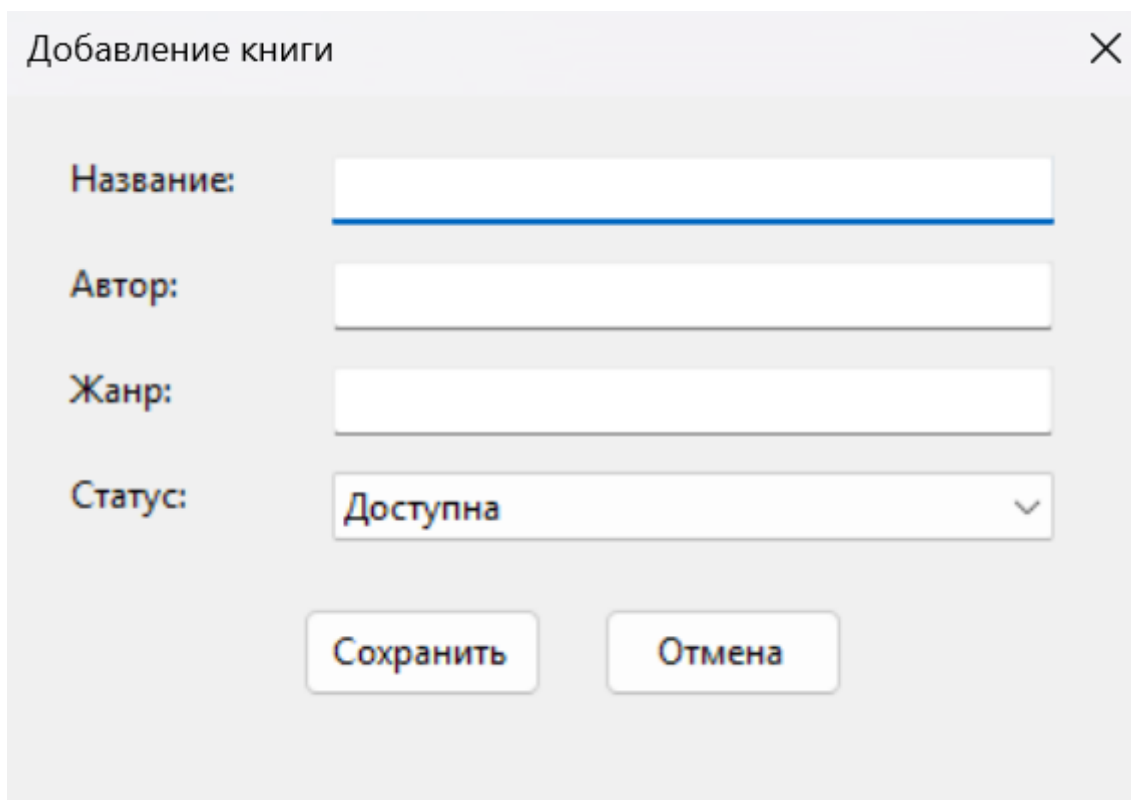
Для хранения данных используется JSON-формат. При добавлении, редактировании или удалении книги вызывается SaveToFile(). При запуске программы данные автоматически загружаются через LoadFromFile()

Далее было создано новое окно для редактирования или создания новой книги.



### 3. Создать второе окно для редактирования или добавления книги

Для редактирования и добавления книг было создано отдельное окно (BookEditForm). Оно содержит поля для ввода названия, автора, жанра и выбора статуса. При нажатии кнопки «Сохранить» данные проверяются и передаются в LibraryManager. Окно открывается как модальное и возвращает результат (DialogResult.OK / Cancel), интерфейс окна показан на рисунке 6.



Добавление книги

Название:

Автор:

Жанр:

Статус:

Сохранить Отмена

Рисунок 6 – Окно для редактирования или добавления книги

Ссылка: 1

```
private void CreateControls()
{
    int y = 20;

    Label lblTitle = new Label { Text = "Название:", Location = new Point(20, y), Size = new Size(80, 25) };
    txtTitle = new TextBox { Location = new Point(110, y), Size = new Size(240, 25) };
    y += 35;

    Label lblAuthor = new Label { Text = "Автор:", Location = new Point(20, y), Size = new Size(80, 25) };
    txtAuthor = new TextBox { Location = new Point(110, y), Size = new Size(240, 25) };
    y += 35;

    Label lblGenre = new Label { Text = "Жанр:", Location = new Point(20, y), Size = new Size(80, 25) };
    txtGenre = new TextBox { Location = new Point(110, y), Size = new Size(240, 25) };
    y += 35;

    Label lblStatus = new Label { Text = "Статус:", Location = new Point(20, y), Size = new Size(80, 25) };
    cmbStatus = new ComboBox
    {
        Location = new Point(110, y),
        Size = new Size(240, 25),
        DropDownStyle = ComboBoxStyle.DropDownList
    };
    cmbStatus.Items.Add("Доступна");
    cmbStatus.Items.Add("Занята");
    cmbStatus.SelectedIndex = 0;
    y += 45;

    btnSave = new Button { Text = "Сохранить", Location = new Point(100, y), Size = new Size(80, 30) };
    btnSave.Click += BtnSave_Click;

    btnCancel = new Button { Text = "Отмена", Location = new Point(200, y), Size = new Size(80, 30) };
    btnCancel.Click += (s, e) => this.DialogResult = DialogResult.Cancel;

    this.Controls.AddRange(new Control[] {
        lblTitle, txtTitle,
        lblAuthor, txtAuthor,
        lblGenre, txtGenre,
        lblStatus, cmbStatus,
        btnSave, btnCancel
    });
}
```

Рисунок 7 – Программное создание элементов управления в окне редактирования

Далее нужно реализовать функционал кнопок, взаимодействующих с книгами. Редактирование, удаление и добавление книг необходимо реализовать для обоих окон.

#### 4. Реализовать редактирование, удаление, добавление книг в обоих окнах

Реализация добавления, редактирования и удаления книг выполняется через методы класса `LibraryManager`, которые вызываются из обработчиков кнопок главного окна.

При нажатии кнопки «Добавить» открывается пустое окно `BookEditForm`. После заполнения полей и сохранения данные передаются в метод `AddBook`, который добавляет книгу в коллекцию, сохраняет изменения в файл и уведомляет интерфейс об обновлении.

```
Ссылка: 1
private void BtnAdd_Click(object sender, EventArgs e)
{
    using (BookEditForm editForm = new BookEditForm())
    {
        if (editForm.ShowDialog() == DialogResult.OK)
        {
            _library.AddBook(editForm.Book);
            UpdateStatus($"Книга \"{editForm.Book.Title}\" добавлена");
        }
    }
}
```

Рисунок 8 – Обработчик кнопки «Добавить» - вызов метода `AddBook`

При нажатии кнопки «Редактировать» открывается то же окно, но с заполненными полями выбранной книги. После сохранения вызывается метод `EditBook`, который обновляет данные книги в коллекции.

```
private void BtnEdit_Click(object sender, EventArgs e)
{
    if (dgvBooks.SelectedRows.Count == 0)
    {
        MessageBox.Show("Выберите книгу для редактирования.", "Внимание", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int selectedId = (int)dgvBooks.SelectedRows[0].Cells["Id"].Value;
    Book book = _library.Books.FirstOrDefault(b => b.Id == selectedId);
    if (book == null) return;

    using (BookEditForm editForm = new BookEditForm(book))
    {
        if (editForm.ShowDialog() == DialogResult.OK)
        {
            _library.EditBook(book.Id, editForm.Book.Title, editForm.Book.Author,
                editForm.Book.Genre, editForm.Book.IsAvailable);
            UpdateStatus($"Книга \"{editForm.Book.Title}\" отредактирована");
        }
    }
}
```

Рисунок 9 – Обработчик кнопки «Редактировать» – вызов метода `EditBook`

Кнопка «Удалить» запрашивает подтверждение у пользователя, а затем

вызывает метод DeleteBook, который удаляет книгу из коллекции.

```
private void BtnDelete_Click(object sender, EventArgs e)
{
    if (dgvBooks.SelectedRows.Count == 0)
    {
        MessageBox.Show("Выберите книгу для удаления.", "Внимание", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int selectedId = (int)dgvBooks.SelectedRows[0].Cells["Id"].Value;
    Book book = _library.Books.FirstOrDefault(b => b.Id == selectedId);
    if (book == null) return;

    DialogResult result = MessageBox.Show($"Удалить книгу \"{book.Title}\"?", "Подтверждение удаления",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        _library.DeleteBook(book.Id);
        UpdateStatus($"Книга \"{book.Title}\" удалена");
    }
}
```

Рисунок 10 – Обработчик кнопки «Удалить» – вызов метода DeleteBook

## 5. Реализовать сохранение актуальной версии списка книг в файл

Для сохранения данных используется формат JSON.

В классе `LibraryManager` реализованы методы `SaveToFile()` и `LoadFromFile()`, которые отвечают за запись и чтение данных из файла `books.json`.

Метод `SaveToFile()` вызывается после каждого изменения коллекции книг (добавление, редактирование, удаление). Он преобразует список книг в JSON-строку с помощью `JsonSerializer.Serialize()` и записывает её в файл.

```
public void SaveToFile()
{
    try
    {
        string json = JsonSerializer.Serialize(_books, new JsonSerializerOptions { WriteIndented = true });
        File.WriteAllText(_saveFilePath, json);
    }
    catch (Exception ex)
    {
        throw new Exception($"Ошибка сохранения файла: {ex.Message}");
    }
}
```

Рисунок 11 – Метод сохранения данных в JSON-файл

Метод `LoadFromFile()` вызывается при запуске программы. Он проверяет наличие файла, читает его содержимое и преобразует JSON-строку обратно в список книг с помощью `JsonSerializer.Deserialize<List<Book>>()`. Если файл отсутствует или повреждён, создаются тестовые книги.

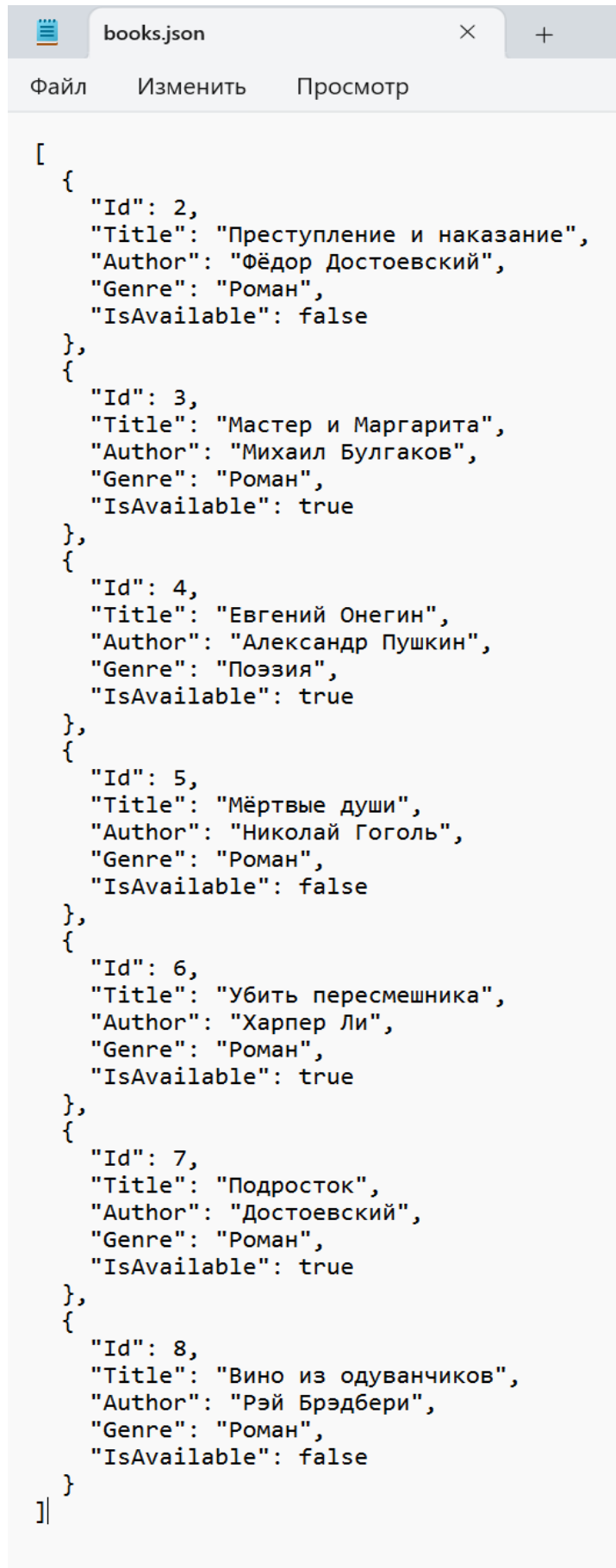
```

public void LoadFromFile()
{
    try
    {
        if (File.Exists(_saveFilePath))
        {
            string json = File.ReadAllText(_saveFilePath);
            var loaded = JsonSerializer.Deserialize<List<Book>>(json);
            if (loaded != null)
            {
                _books.Clear();
                foreach (var book in loaded)
                {
                    _books.Add(book);
                }
            }
            else
            {
                AddTestBooks();
            }
        }
        catch (Exception)
        {
            _books.Clear();
            AddTestBooks();
        }
    }
}

```

Рисунок 12 – Метод загрузки данных из JSON-файла

Использование JSON-формата обеспечивает лёгкость чтения и редактирования данных вручную, а также совместимость с различными системами.



```
[
  {
    "Id": 2,
    "Title": "Преступление и наказание",
    "Author": "Фёдор Достоевский",
    "Genre": "Роман",
    "IsAvailable": false
  },
  {
    "Id": 3,
    "Title": "Мастер и Маргарита",
    "Author": "Михаил Булгаков",
    "Genre": "Роман",
    "IsAvailable": true
  },
  {
    "Id": 4,
    "Title": "Евгений Онегин",
    "Author": "Александр Пушкин",
    "Genre": "Поэзия",
    "IsAvailable": true
  },
  {
    "Id": 5,
    "Title": "Мёртвые души",
    "Author": "Николай Гоголь",
    "Genre": "Роман",
    "IsAvailable": false
  },
  {
    "Id": 6,
    "Title": "Убить пересмешника",
    "Author": "Харпер Ли",
    "Genre": "Роман",
    "IsAvailable": true
  },
  {
    "Id": 7,
    "Title": "Подросток",
    "Author": "Достоевский",
    "Genre": "Роман",
    "IsAvailable": true
  },
  {
    "Id": 8,
    "Title": "Вино из одуванчиков",
    "Author": "Рэй Брэдбери",
    "Genre": "Роман",
    "IsAvailable": false
  }
]
```

Рисунок 13 – Пример содержимого файла books.json с сохранёнными книгами

## 6. Проверить программу на наличие ошибок, корректную работу исключений

В процессе тестирования были проверены сценарии, которые могут привести к ошибкам:

- Попытка добавить книгу без названия
- Попытка добавить книгу без автора
- Попытка добавить книгу без жанра
- Попытка удалить невыбранную книгу

Во всех случаях программа не падает, а выводит понятные сообщения об ошибках через MessageBox.

Также обрабатываются ошибки при работе с файлом books.json. Если файл повреждён или отсутствует, программа создаёт тестовые книги и продолжает работу

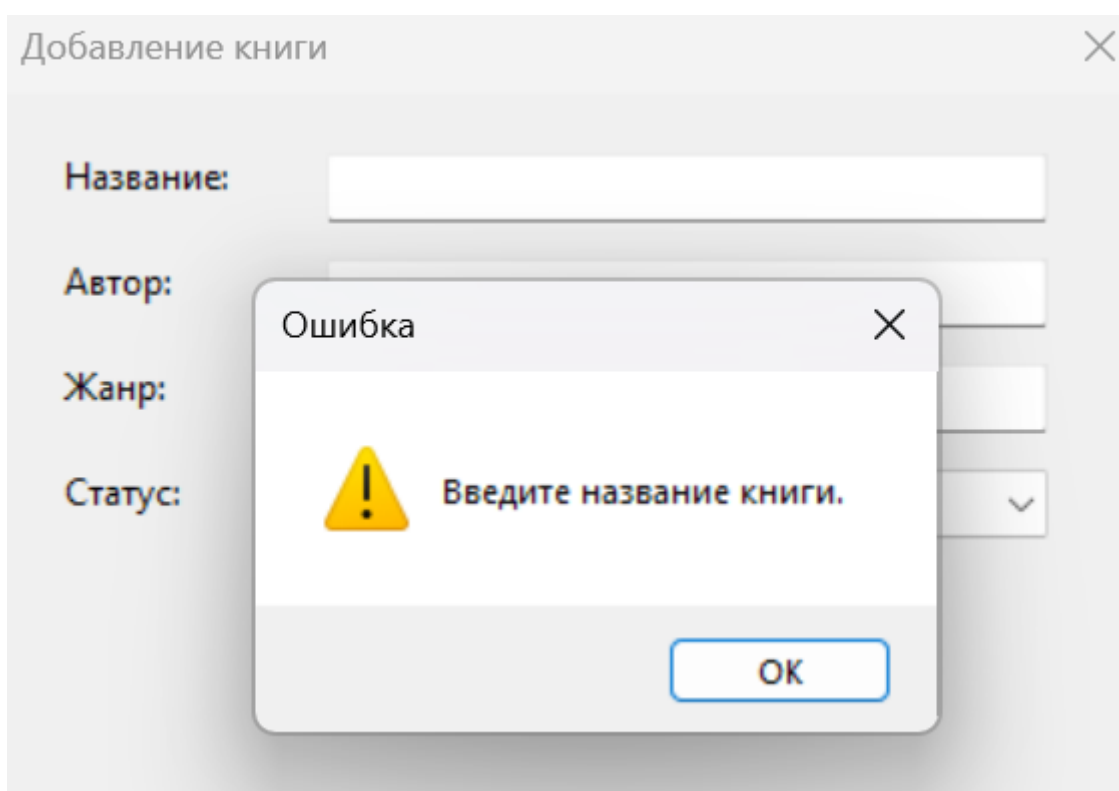


Рисунок 14 – Сообщение об ошибке при попытке добавить книгу без названия



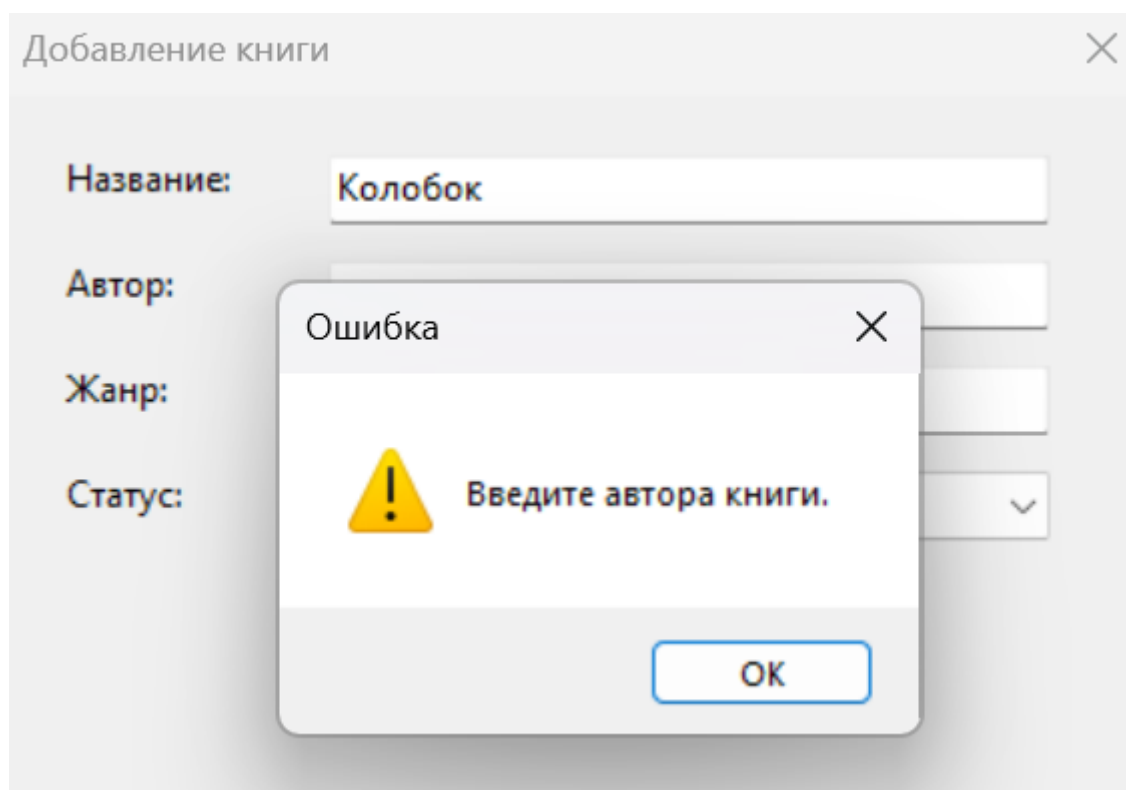


Рисунок 15 – Сообщение об ошибке при попытке добавить книгу без автора

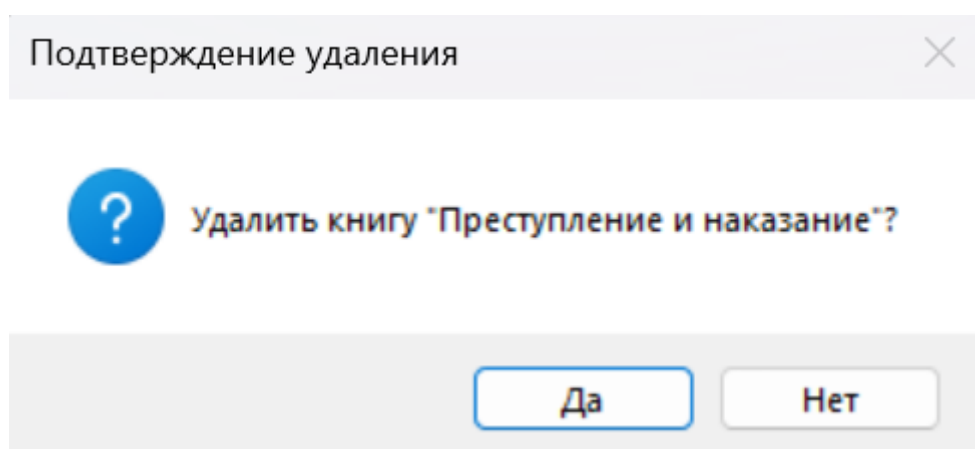


Рисунок 16 – Подтверждение удаления книги

## **ЗАКЛЮЧЕНИЕ**

При выполнении этого задания было создано приложение для управления библиотекой, которое позволит пользователям добавлять, редактировать и удалять книги, а также отслеживать информацию о них. Все основные требования выполнены, программа корректно выполняет указанные функции. Ход работы отражён в отчёте, функциональность протестирована.

В процессе разработки были использованы следующие технологии: Windows Forms, LINQ для фильтрации, поиска и сортировки данных, события для автоматического обновления интерфейса, пространство имён System.IO для работы с файлами, интерфейс INotifyPropertyChanged для уведомлений об изменениях, а также System.Text.Json для сохранения и загрузки данных в формате JSON.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) СТУ 7.5-07.2021. Общие требования к построению, изложению и оформлению документов учебной деятельности: взамен СТО 4.2-07-2014: дата введения 2021-12-30. – Красноярск. – 61с.
- 2) Сайт METANIT [Электронный ресурс] – <https://metanit.com/sharp/windowsforms/>
- 3) Microsoft Как записать объекты .NET в формате JSON (сериализация) [Электронный ресурс] – <https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/how-to>
- 4) Система электронного обучения СФУ: официальный сайт – 2017. – Режим доступа: URL: <https://e.sfu-kras.ru/>